# Il percorso di PharmaBI

Origine del progetto, uso dell’AI e presentazione del proprio livello di autonomia. Appunti aggiornati al 6 novembre 2026.

## Perché nasce il progetto

PharmaBI nasce come percorso di apprendimento della data analysis, applicato ai dati della parafarmacia che Danilo co-gestisce. La finalità pratica è comprendere l’andamento delle vendite e dei prodotti per supportare le decisioni sull’attività. Con il tempo il progetto è cresciuto in ampiezza e complessità.

## Apertura della presentazione concordata

Ho iniziato PharmaBI come percorso di apprendimento della data analysis, applicato ai dati della parafarmacia che co-gestisco. Volevo imparare a leggere meglio l’andamento delle vendite e dei prodotti, per supportare le decisioni sull’attività.

Nel tempo il progetto è cresciuto, fino a comprendere una pipeline di elaborazione dei dati, controlli di qualità e dashboard interattive.

Per sviluppare il codice ho utilizzato l’assistenza dell’AI e sto ancora costruendo la mia autonomia in R e SQL. La versione che presento utilizza dati sintetici indipendenti, così posso mostrare il lavoro senza divulgare i dati dell’attività.

## Come descrivere il proprio contributo

Il contributo comprende la conoscenza dell’attività, le domande da affrontare e l’interpretazione dei risultati. Programmazione e costruzione dei controlli hanno ricevuto assistenza AI. Il risultato mostrato va accompagnato da una descrizione sincera di ciò che si sa spiegare, modificare o riprodurre autonomamente e di ciò per cui serve ancora supporto.

**Metodo di studio:** per ogni parte chiedersi quale problema affronta, che cosa rappresentano i dati e come si controlla il risultato. I controlli superati offrono evidenza sulle condizioni verificate, ma non garantiscono da soli la correttezza di ogni aspetto del progetto.

Ambito: introduzione al portfolio, granularità, domande sul modello e verifiche dei risultati. Le tabelle didattiche usano dati inventati. I controlli eseguiti sono distinti dalle prove ancora da svolgere.

# Che cosa rappresenta una riga

Granularità, chiave logica e differenza tra righe, scontrini, prodotti e confezioni.

## La granularità della tabella

La **granularità** è il livello di dettaglio di una riga. Nella fact_sales di PharmaBI una riga rappresenta una combinazione di **scontrino e codice prodotto**. La coppia sale_id + product_code è la chiave logica di cui verifichiamo l'unicità.

| Scontrino | Prodotto | Confezioni |
|---|---|---|
| S001 | A | 2 |
| S001 | B | 1 |
| S002 | A | 1 |

In questo esempio ci sono **3 righe, 2 scontrini distinti, 2 prodotti distinti e 4 confezioni**. Le due confezioni di A nello scontrino S001 occupano una riga sola: la quantità è 2.

## Quali ripetizioni sono normali?

| Situazione | Esempio | Significato |
|---|---|---|
| Stesso scontrino, prodotti diversi | S001 + A<br>S001 + B | Normale: uno scontrino contiene più prodotti. |
| Stesso prodotto, scontrini diversi | S001 + A<br>S002 + A | Normale: il prodotto compare in più vendite. |
| Stessa coppia in due righe | S001 + A<br>S001 + A | Da indagare: la coppia non è più unica. |

## Perché non basta una sola colonna?

sale_id si ripete quando uno scontrino contiene più prodotti. product_code si ripete quando il prodotto compare su scontrini diversi. La combinazione distingue le righe nel modello scelto. Una vendita di due confezioni non richiede due righe, se la quantità totale è registrata nella stessa riga.

## Attenzione al significato dell'identificativo

Nel gestionale della parafarmacia il numero dello scontrino è progressivo e non ricomincia cambiando anno, secondo l'esperienza operativa riferita da Danilo. Questa regola riguarda l'identificazione dello scontrino. Il modo in cui il gestionale tratta due righe dello stesso prodotto è una proprietà separata. Due scontrini non equivalgono necessariamente a due clienti distinti.

# Coppia ripetuta: possibile errore

Prima ipotesi: la stessa registrazione di vendita è entrata due volte nei dati.

Guardiamo due righe con lo stesso scontrino e lo stesso prodotto. In questo esempio il prezzo netto unitario è 5,00 euro e ciascuna riga contiene due confezioni, quindi il ricavo di riga è 10,00 euro.

| Riga | Scontrino | Prodotto | Confezioni | Ricavo EUR |
|---|---|---|---|---|
| 1 | S001 | A | 2 | 10,00 |
| 2 | S001 | A | 2 | 10,00 |

## Perché la coppia è ripetuta?

La chiave candidata è S001 + A nella riga 1 ed è ancora S001 + A nella riga 2. Cambia la posizione nel file, ma non cambia la coppia che dovrebbe identificare la registrazione. Un numero di riga aggiunto dopo l’importazione non prova che siano due vendite diverse.

## Se la seconda riga è una copia accidentale

Ipotizziamo che il gestionale confermi una sola registrazione da due confezioni e 10,00 euro. Se sommiamo entrambe le righe del file importato, otteniamo quattro confezioni e 20,00 euro. Il numero di scontrini distinti rimane invece 1. Un indicatore può quindi sembrare corretto mentre altri sono raddoppiati.

| Indicatore | Se la riga conta una volta | Se conta due volte |
|---|---|---|
| Righe | 1 | 2 |
| Scontrini distinti | 1 | 1 |
| Confezioni | 2 | 4 |
| Ricavo EUR | 10,00 | 20,00 |

## Come si può arrivare a questa situazione?

Per esempio, lo stesso file viene caricato due volte oppure si concatenano esportazioni che coprono periodi sovrapposti. Un’altra possibile causa è una join che moltiplica le righe perché trova più corrispondenze nella tabella collegata.

**Due righe identiche sono un indizio, non una prova definitiva.** Potrebbero anche rappresentare due registrazioni reali identiche. Per distinguere i casi servono il dato di origine, eventuali identificativi della riga e un confronto con i totali attesi. Se la duplicazione accidentale è confermata, si corregge la causa e si ricontrollano i risultati.

# Coppia ripetuta: righe legittime

Esempio ipotetico: due righe dello stesso prodotto, con prezzi diversi. Questo comportamento resta da verificare nel gestionale utilizzato.

Un cliente acquista due confezioni di A: una a prezzo netto 9,00 euro, una a 7,00 euro. Il gestionale potrebbe registrarle separatamente. Qui la coppia S001 + A si ripete, ma entrambe le righe descrivono una vendita reale.

| Riga origine | Scontrino | Prodotto | Confezioni | Prezzo netto unitario EUR | Ricavo EUR |
|---|---|---|---|---|---|
| L01 | S001 | A | 1 | 9,00 | 9,00 |
| L02 | S001 | A | 1 | 7,00 | 7,00 |

## Opzione A: una riga per scontrino e prodotto

Se l’analisi richiede il totale del prodotto nello scontrino, possiamo aggregare le due righe. Le confezioni diventano 2 e il ricavo diventa 16,00 euro. Occorre una regola esplicita per ogni altro campo: non basta scegliere la prima riga.

| Scontrino | Prodotto | Confezioni | Ricavo EUR |
|---|---|---|---|
| S001 | A | 2 | 16,00 |

Se il costo registrato è 5,00 euro per ciascuna confezione, il costo aggregato è 10,00 euro e il profitto è 6,00 euro. Il margine aggregato è 6 / 16 = **37,50%**. Le quantità e gli importi di riga si sommano; le percentuali si ricalcolano sui totali.

## Opzione B: conservare il dettaglio della riga

Se interessa distinguere il prezzo applicato o lo sconto, possiamo conservare L01 e L02. La granularità diventa la **riga dello scontrino**. Una possibile chiave è sale_id + line_id, se line_id identifica univocamente la riga all’interno dello scontrino. product_code resta una caratteristica della riga e può ripetersi.

## La conseguenza sul modello

Conservare il dettaglio non è coerente con un controllo che impone una sola riga per sale_id + product_code. Va quindi aggiornata la definizione della granularità e adeguato il controllo. La scelta dipende dalle domande analitiche e dalle informazioni disponibili nella fonte.

**Eliminare una riga legittima perderebbe una vendita. Sommare una copia accidentale gonfierebbe i risultati.** Sono due problemi diversi che possono presentarsi con la stessa coppia ripetuta.

# Come indagare una coppia ripetuta

Il controllo segnala che la chiave prevista non è unica. La causa si determina esaminando le registrazioni.

## 1. Isolare il gruppo di righe

Si raggruppano le registrazioni per sale_id e product_code e si individuano i gruppi con più di una riga. Questo controllo individua la violazione della chiave candidata, anche quando prezzi o quantità sono diversi. Non cerca soltanto righe interamente identiche.

## 2. Confrontare tutti i campi utili

Si confrontano quantità, ricavi, costi, orari ed eventuali identificativi della riga originale. Differenze nei campi possono indicare registrazioni distinte. L'uguaglianza di tutti i campi richiede comunque di verificare la provenienza.

## 3. Risalire alla fonte e alla trasformazione

Si controllano esportazione originale, periodi caricati e passaggi che combinano tabelle. Se la coppia è unica prima di una join e si ripete dopo, la join è un punto preciso da indagare.

| Prima della join | Tabella collegata | Possibile risultato |
|---|---|---|
| Una riga di vendita S001 + A | Due righe per il codice A | La vendita trova due corrispondenze e produce due righe. |

Se ci si aspetta una sola classificazione per prodotto, va definita una regola per rendere univoca la tabella collegata. Per classificazioni storiche può servire anche una condizione sulla data. Non si sceglie una corrispondenza a caso.

## 4. Decidere e documentare la regola

Duplicazione accidentale confermata: correggere la causa. Registrazioni reali da riassumere: aggregare secondo regole esplicite. Dettaglio da conservare: ridefinire la granularità e la chiave. Se le informazioni non bastano, segnalare il caso e cercare chiarimenti sulla fonte.

## 5. Ripetere i controlli pertinenti

Dopo l'intervento si ricontrollano unicità della chiave prevista, quantità e importi, numero di scontrini e formule economiche. I totali attesi devono essere coerenti con la fonte e con la decisione documentata.

**Nel dataset di PharmaBI abbiamo verificato l'unicità di sale_id + product_code.** Gli esempi di coppie ripetute di queste pagine sono ipotesi didattiche: non indicano che il problema sia presente nei dati già validati.

# Domande possibili al colloquio

Tracce di risposta da comprendere e riformulare con parole proprie.

## Domanda principale. Qual è la granularità e perché?

Una riga rappresenta un prodotto all'interno di uno scontrino. Questo livello permette di studiare sia le vendite dei prodotti sia la composizione degli scontrini. La scelta deve essere compatibile con la struttura dei dati disponibili e con le analisi previste.

## 1. Perché non basta lo scontrino o il prodotto?

Uno scontrino può contenere più prodotti e un prodotto può comparire in molti scontrini. Le singole colonne si ripetono normalmente. Nel modello scelto la combinazione delle due deve essere unica.

## 2. Come hai verificato l'unicità?

È presente un controllo che raggruppa per sale_id e product_code e cerca gruppi con più di una riga. Abbiamo eseguito il controllo sui dati del progetto. Anche lo sviluppo dei controlli ha ricevuto assistenza AI.

## 3. Che cosa faresti con due righe della stessa coppia?

Prima confronterei le righe e risalirei alla fonte. Se fossero la stessa registrazione importata due volte, correggerei la duplicazione. Se fossero due registrazioni reali, valuterei se aggregarle oppure conservare il dettaglio con un identificativo della riga. Poi verificherei chiave e totali. Non eliminerei una riga soltanto perché la coppia si ripete.

## 4. Come influisce la granularità sugli indicatori?

Il numero di righe conta le registrazioni al livello scelto. Gli scontrini si contano con i sale_id distinti, i prodotti con i product_code distinti e le confezioni sommando le quantità. Se la granularità cambia, il significato del conteggio delle righe cambia.

## 5. Il numero dello scontrino è sempre univoco?

Nel gestionale che utilizzo il numero è progressivo anche tra anni diversi, quindi identifica scontrini diversi senza ricominciare a ogni anno. Questa regola non chiarisce da sola come il gestionale registri lo stesso prodotto a due prezzi nello stesso scontrino: quel comportamento resta da verificare. Per altre fonti, l'unicità dell'identificativo va valutata nel loro specifico contesto.

**Se una parte supera l'autonomia attuale:** dichiarare che è stata sviluppata con assistenza e spiegare il concetto che si è compreso. Il portfolio deve rappresentare il percorso di apprendimento e il livello effettivo di competenza.

# Ripasso e autoverifica

Tre esercizi brevi e un glossario per fissare i concetti.

## Esercizio 1. Contare senza confondere i livelli

Un solo scontrino contiene A con quantità 3 e B con quantità 2. La tabella ha una riga per scontrino e prodotto. Quante sono le righe, gli scontrini, i prodotti distinti e le confezioni?

## Esercizio 2. Interpretare una coppia ripetuta

Trovi due righe S010 + C, ciascuna con quantità 1 e ricavo 8,00 euro. Le righe sono uguali. Puoi già concludere che una delle due vada eliminata? Quale informazione cercheresti?

## Esercizio 3. Conservare due registrazioni reali

Lo scontrino S020 contiene due righe legittime dello stesso prodotto D, una da 6,00 euro e una da 4,00 euro. Qual è il totale se le aggreghi? Se vuoi conservare i due prezzi, quale informazione ti serve per distinguere le righe?

## Risposte ragionate

**1.** Due righe, uno scontrino, due prodotti distinti e cinque confezioni.
**2.** No. Occorre capire se sono due registrazioni reali o una copia della stessa registrazione. Si cercano fonte, identificativi di riga e totali di confronto.
**3.** Due confezioni e 10,00 euro di ricavo, supponendo una confezione per riga. Per conservare i due prezzi serve un identificativo affidabile della riga, per esempio line_id univoco all’interno dello scontrino.

## Glossario essenziale

| Termine | Significato in questo contesto |
|---|---|
| Granularità | Che cosa rappresenta una singola riga della tabella. |
| Chiave logica | Campo o combinazione di campi che deve identificare una riga secondo le regole del modello. |
| Coppia ripetuta | Due o più righe hanno gli stessi valori di sale_id e product_code. |
| Aggregare | Riunire più registrazioni secondo una regola, per esempio sommando quantità e importi. |
| Join | Collegare tabelle tramite condizioni di corrispondenza. Più corrispondenze possono moltiplicare le righe. |
| Pipeline | Sequenza di passaggi che importa, trasforma e controlla i dati. |

Materiale elaborato dalla discussione del 6 novembre 2026. Gli esempi usano dati inventati, con importi netti in euro. Questo quaderno raccoglie l’approfondimento attuale e potrà essere esteso durante la preparazione del portfolio.

# Regole confermate e punti aperti

La conoscenza del gestionale aiuta a formulare le verifiche, distinguendo esperienza operativa, controlli sui dati e ipotesi.

| Aspetto | Stato attuale |
|---|---|
| Numero dello scontrino | Danilo riferisce un progressivo continuo, indipendente dall’anno. Scontrini diversi hanno numeri diversi. |
| Chiave scontrino + prodotto | Il controllo di unicità è stato superato nei dati analizzati. Questo risultato descrive quei dati. |
| Stesso prodotto a due prezzi | Caso possibile, per esempio una confezione in scadenza scontata e una a prezzo pieno. Danilo non lo ha osservato in tre anni. |
| Risultato nell’esportazione | Da verificare: il gestionale potrebbe mantenere due righe oppure accorpare quantità e importi. |

## Che cosa non possiamo ancora concludere

La rarità del caso non dimostra che sia impossibile. L’assenza di coppie ripetute nei dati attuali non garantisce che la stessa regola varrà per ogni esportazione futura. È corretto mantenere il controllo e indagare eventuali nuove occorrenze, senza attribuirle automaticamente a una trasformazione errata.

## Che cosa osservare in una futura verifica

In un’eventuale prova in ambiente di test, o quando si osserverà una vendita reale di questo tipo, è utile confrontare lo scontrino con l’esportazione originale. Le domande sono: quante righe produce per quel codice prodotto? Quale quantità registra? Come rappresenta gli importi? Il totale corrisponde alle due confezioni vendute?

Se il prezzo netto è 9,00 euro per una confezione e 7,00 euro per l’altra, il totale atteso è 2 confezioni e 16,00 euro. Se compare un solo valore di prezzo, bisogna capire se rappresenta un prezzo medio, un totale o altro. L’ipotesi non va confusa con il comportamento effettivamente osservato.

## Una risposta coerente al colloquio

Nel mio gestionale il numero dello scontrino è progressivo e univoco anche tra anni diversi. Nei dati analizzati ho verificato anche l’unicità della coppia scontrino-prodotto. Per il caso raro dello stesso prodotto a due prezzi non ho ancora verificato come il gestionale costruisca l’esportazione: lo considero un punto da approfondire.

Questa precisazione non richiede di modificare il modello attuale. Se una futura verifica mostrasse un dettaglio diverso nella fonte, valuteremmo la regola di trasformazione e i controlli pertinenti.

# Verifiche effettivamente eseguite

Evidenze raccolte sul progetto sviluppato con assistenza AI. Ogni controllo ha uno scopo preciso e un ambito limitato.

## 1. Struttura e regole della fact table

Abbiamo eseguito controlli su presenza delle colonne richieste, tipi, campi obbligatori, testi vuoti e valori economici non finiti. Sono presenti anche verifiche sull'unicità di sale_id + product_code, sulle quantità intere e sulla coerenza della data e del tipo di vendita all'interno dello scontrino. Questi controlli confrontano i dati con le regole dichiarate del modello.

## 2. Coerenza delle formule economiche

Il validatore confronta il profitto con ricavi meno costi e, per le righe con ricavo diverso da zero, il margine con profitto diviso ricavo. Usa una tolleranza numerica di 1e-10. Un valore negativo o un costo zero viene esaminato negli audit, senza essere automaticamente eliminato. Una formula coerente non prova che ogni valore originario del gestionale sia corretto.

## 3. Confronto tra RDS e SQLite

Lo script della demo confronta tutte le righe e le colonne della fact table nel database con il file RDS preparato. Ordina le righe per chiave e confronta anche i valori, non soltanto il totale dei ricavi. Confronta inoltre la vista retail con le righe retail attese. Questo aiuta a individuare perdite, aggiunte o alterazioni durante il trasferimento.

## 4. Ricalcolo di risultati in R e SQL

Totali per tipo di vendita, dati mensili, medie per giorno attivo e analisi per intervallo vengono calcolati attraverso percorsi distinti e confrontati. Per i panieri vengono confrontati anche i risultati a livello di scontrino. L'accordo offre una verifica incrociata, ma due implementazioni possono condividere una regola concettualmente sbagliata.

## 5. Dimensioni e file esportati

Le descrizioni dei prodotti vengono confrontate con il RDS, il calendario con le date calcolate per il 2026 e la dimensione ATC di livello 2 con il riferimento disponibile. I quattro CSV per Power BI vengono riletti, controllando colonne, conversioni numeriche e valori rispetto al database. L'export CSV verificato non equivale a verificare automaticamente ogni misura DAX.

Fonti nel progetto: R/validation/validate_fact_sales.R, R/demo/04_verify_demo.R e docs/demo_verification.md. I confronti numerici dello script 04 usano una tolleranza assoluta di 1e-7, distinta da quella delle formule economiche.

# Casi particolari e prove pratiche

Il comportamento va osservato anche fuori dal caso ordinario di un periodo completo con molte vendite.

## Intervalli temporali diversi

Il verificatore confronta R e SQL su sette intervalli: l'intera finestra della demo, febbraio, marzo, un periodo a cavallo di due mesi, un solo giorno, due giorni di chiusura fittizia e un periodo fuori dalla copertura. Le date estreme sono incluse nel filtro.

Per gli intervalli senza vendite controlla conteggi pari a zero, aggregati monetari mancanti e assenza di righe mensili, scontrini e coppie. Questo evita di confondere assenza di dati e vendite effettivamente pari a zero. È una prova di un caso limite reale nella demo, diversa dall'inserire volutamente dati errati per mettere alla prova un validatore.

## Coppie di prodotti: due metodi di calcolo

In R lo script costruisce le combinazioni di prodotti dentro ciascuno scontrino e le confronta con la query SQL. Il confronto comprende frequenze, supporto, confidence nelle due direzioni e lift, oltre alle descrizioni e ai conteggi degli scontrini contenenti i singoli prodotti.

Controlla anche una relazione matematica: uno scontrino con k prodotti distinti genera k × (k - 1) / 2 coppie non ordinate. Con tre prodotti A, B e C le coppie sono A-B, A-C e B-C, quindi 3. La somma delle frequenze delle coppie deve coincidere con la somma di questi contributi per tutti gli scontrini.

## Uso manuale delle dashboard

Durante il lavoro sono stati provati filtri temporali, cambi di metrica e aggiornamento delle visualizzazioni. Per Shiny sono stati verificati anche l'intervallo vuoto e il ritorno a un periodo con dati. Le prove in interfaccia sono manuali: non fanno parte dei 70 controlli automatici e non dimostrano che ogni possibile combinazione di filtri sia stata provata.

## Esecuzione su Windows e Linux

Il verificatore della demo ha completato 70 controlli con esito Passed su entrambi i sistemi. Anche i tre casi analitici sono stati eseguiti sui due PC con risultati coerenti. Questo documenta le esecuzioni effettuate negli ambienti disponibili, senza implicare una verifica di macOS o di ogni possibile installazione.

Riscontri disponibili: output condivisi durante il lavoro, docs/demo_verification.md e report locali in data/demo/validation/run-*. La verifica del comportamento visuale e delle misure Power BI resta distinta dai confronti automatici sui CSV.

# Che cosa possiamo affermare

Una descrizione credibile indica sia l’evidenza raccolta sia ciò che deve ancora essere verificato.

## I 70 Passed: che cosa significano

Significano che le condizioni implementate nel verificatore hanno avuto esito positivo sulla demo esaminata. Non rappresentano una percentuale di correttezza, una certificazione indipendente o una prova di tutti i possibili comportamenti del software. Alcuni confronti riassumono molte righe, mentre altri controllano una singola condizione.

## Baseline, impronte dei file e report

Il confronto con i totali documentati della demo segnala cambiamenti rispetto allo scenario atteso. È una verifica di regressione, non un totale esterno calcolato indipendentemente dal generatore. Le impronte MD5 confrontate prima e dopo l’esecuzione rilevano modifiche ai file di input, senza dimostrarne la correttezza. I report registrano gli esiti e l’ambiente di verifica, rendendo consultabili le evidenze.

## Il ruolo dell’AI nella verifica

Anche il codice dei controlli è stato sviluppato con assistenza AI. Far riscrivere la stessa verifica all’AI non elimina la possibilità di un errore condiviso. Le evidenze utili sono i confronti osservati, le regole comprensibili e i riferimenti usati. Per ciascun risultato bisogna poter indicare quale verifica lo sostiene e quale errore potrebbe ancora sfuggire.

## Prove dei controlli: quattro scenari verificati

Il 6 novembre, su Linux, abbiamo provato la demo integra e tre copie modificate separatamente: una riga duplicata, un profitto errato e un margine errato. Il caso integro è stato accettato e ciascuna copia rifiutata nel punto previsto. Abbiamo poi raccolto i quattro scenari in uno script ed eseguito nuovamente il file dopo il riavvio di R, con quattro Passed. Queste prove non coprono ogni controllo; restano possibili altri casi, per esempio sui campi obbligatori vuoti.

## Una formulazione per il recruiter

Ho sviluppato il progetto e i controlli con assistenza AI. Ho eseguito verifiche strutturali ed economiche, confronti tra RDS, SQLite e CSV e ricalcoli in R e SQL, compresi periodi senza vendite. Ho provato le dashboard e ripetuto le verifiche su Windows e Linux. Ho anche messo alla prova il validatore sulla demo integra e su copie con errori introdotti apposta: una chiave ripetuta, un profitto errato e un margine errato. Ho osservato l’accettazione del caso integro e il rifiuto degli altri nel punto previsto.

Stato del percorso al 6 novembre 2026. Queste note distinguono le verifiche già eseguite dalle attività proposte. Non è stata svolta una certificazione esterna o una revisione indipendente dell’intero codice.

# Prova del validatore: caso integro

Esecuzione in console su Linux il 6 novembre 2026. La prova usa la funzione del progetto, senza riscriverne una copia semplificata.

## La regola da mettere alla prova

La coppia sale_id + product_code deve essere unica nella fact table. Il risultato atteso sul dataset demo integro è una validazione completa. Il frammento seguente mostra il controllo presente nella funzione validate_fact_sales(), che carica il proprio oggetto fact_sales dal file RDS.

```
duplicate_keys <- fact_sales %>%
  count(
    sale_id,
    product_code,
    name = "rows_per_key"
  ) %>%
  filter(rows_per_key > 1)

stopifnot(nrow(duplicate_keys) == 0)
```

count() conta quante righe condividono ciascuna coppia. filter() conserva i gruppi con più di una riga. stopifnot() interrompe l’esecuzione se non è vero che i gruppi ripetuti siano zero.

## Codice eseguito per il caso integro

```
source(here::here("R", "config", "data_paths.R"))
source(
  here::here("R", "validation", "validate_fact_sales.R")
)

validator_test_paths <- pharmabi_data_paths("demo")
validator_test_valid <- validate_fact_sales(
  validator_test_paths$fact_file
)

print(validator_test_valid$validation_summary, width = Inf)
```

## Risultato osservato

Il riepilogo ha mostrato **14 controlli Passed**, compresa l’unicità della chiave con zero coppie duplicate. Il dataset conteneva 7.835 righe. Lo scarto massimo sulla formula del profitto era 5.684342e-14, inferiore alla tolleranza prevista di 1e-10. La formula del margine aveva scarto zero.

**Che cosa dimostra:** il validatore accetta questo input integro. Per verificare che riconosca una chiave ripetuta serve il caso della pagina successiva.

# Prova del validatore: coppia ripetuta

Una modifica controllata al dataset: aggiunta di una copia esatta della prima riga. Risultato atteso: rifiuto sul controllo della chiave.

## Creazione della copia e file temporaneo

```
validator_test_duplicate <- dplyr::bind_rows(
  validator_test_valid$fact_sales,
  validator_test_valid$fact_sales[1, ]
)

validator_test_duplicate_file <- tempfile(
  pattern = "pharmabi_duplicate_",
  fileext = ".rds"
)

saveRDS(
  validator_test_duplicate,
  validator_test_duplicate_file
)
```

bind_rows() aggiunge una riga identica alla prima. La copia ha 7.836 righe, anziché 7.835, e una combinazione ripetuta di scontrino e prodotto. È salvata nel percorso temporaneo creato da tempfile(); il comando non sovrascrive il file demo originale.

## Esecuzione del validatore effettivo

```
validate_fact_sales(
  validator_test_duplicate_file
)
```

## Esito osservato nella console

```
Checking logical-key uniqueness...
Errore in validate_fact_sales(validator_test_duplicate_file) :
  nrow(duplicate_keys) == 0 non è TRUE
```

Prima dell’interruzione risultavano superati i controlli su tipi, campi obbligatori, testi, valori economici finiti, ATC e metadati dello scontrino. L’errore indica proprio l’asserzione sulla chiave: non è un’interruzione generica attribuita automaticamente alla duplicazione.

| Caso | Atteso | Osservato |
|---|---|---|
| Demo integra | Validazione completa | 14 Passed |
| Copia con una riga ripetuta | Rifiuto per chiave non unica | Interruzione su duplicate_keys |

**Conclusione:** entrambe le prove hanno prodotto l’esito previsto. Il controllo riconosce questa duplicazione introdotta appositamente. Il risultato non estende automaticamente la prova agli altri controlli o a ogni possibile input.

Se compare Browse[1]>, la console è nel debugger. Per uscire da quel contesto si può digitare Q e premere Invio. Il prompt di debug non cambia il motivo dell’errore osservato.

# Profitto errato: prova mirata

Risultato atteso: il validatore deve rifiutare un profitto che non corrisponde a ricavi meno costi. Si riparte sempre dal dataset integro.

## Preparazione del caso

```
validator_test_profit <- validator_test_valid$fact_sales

stopifnot(validator_test_profit$revenue[1] != 0)

validator_test_profit$profit[1] <-
  validator_test_profit$profit[1] + 1

validator_test_profit$margin[1] <-
  validator_test_profit$profit[1] /
  validator_test_profit$revenue[1]
```

Il profitto della prima riga aumenta di 1 euro, mentre ricavi e costi restano invariati. Si ricalcola il margine per mantenerlo coerente con il nuovo profitto dichiarato. Vengono quindi modificati due campi, ma si introduce una sola incoerenza tra le formule: profitto diverso da ricavi meno costi.

## Salvataggio temporaneo ed esecuzione

```
validator_test_profit_file <- tempfile(
  pattern = "pharmabi_profit_",
  fileext = ".rds"
)

saveRDS(validator_test_profit, validator_test_profit_file)

validate_fact_sales(validator_test_profit_file)
```

## Esito osservato

La console ha confermato i controlli su tipi, valori mancanti, testi, valori economici finiti, ATC, metadati, unicità della chiave e quantità. Dopo Checking financial consistency... l’esecuzione si è interrotta sulla condizione seguente:

```
financial_validation$max_profit_difference < financial_tolerance
non è TRUE
```

Questo è il rifiuto atteso per la formula del profitto. Non stiamo considerando sufficiente un errore qualsiasi: il messaggio identifica l’asserzione collegata alla modifica effettuata. Il controllo del margine non è dichiarato superato da questo output, perché l’interruzione avviene prima della relativa asserzione.

## Perché una nuova copia per ogni prova?

Ripartire da validator_test_valid$fact_sales evita di portare nel nuovo scenario la duplicazione della prova precedente. In quel caso il validatore si fermerebbe prima sulla chiave e non raggiungerebbe il controllo economico che vogliamo esaminare.

# Margine errato e riepilogo

Risultato atteso: rifiuto sul margine, mantenendo corretti ricavi, costi e profitto. Esito osservato il 6 novembre 2026 su Linux.

## Codice eseguito

```
validator_test_margin <- validator_test_valid$fact_sales

validator_test_margin$margin[1] <-
  validator_test_margin$margin[1] + 0.10

validator_test_margin_file <- tempfile(
  pattern = "pharmabi_margin_",
  fileext = ".rds"
)

saveRDS(validator_test_margin, validator_test_margin_file)

validate_fact_sales(validator_test_margin_file)
```

Il margine è espresso come proporzione. Aggiungere 0.10 significa aggiungere 10 punti percentuali. Qui viene modificato soltanto il margine della prima riga. Ricavi, costi, profitto e chiavi restano quelli del dataset integro.

## Esito osservato

Dopo i controlli strutturali, quelli sulla chiave e sulle quantità, la validazione ha raggiunto la sezione economica. L’interruzione è avvenuta sulla condizione:

```
margin_validation$max_margin_difference < financial_tolerance
non è TRUE
```

Nella funzione, l’asserzione sulla formula del profitto precede quella sul margine. Il messaggio osservato corrisponde quindi al rifiuto atteso sul margine alterato.

## Riepilogo delle quattro prove

| Scenario | Risultato atteso | Risultato osservato |
|---|---|---|
| Demo integra | Validazione completa | 14 controlli Passed |
| Copia con una riga duplicata | Rifiuto sulla chiave | Errore su duplicate_keys |
| Copia con profitto alterato | Rifiuto sulla formula del profitto | Errore su max_profit_difference |
| Copia con margine alterato | Rifiuto sulla formula del margine | Errore su max_margin_difference |

**Conclusione:** i quattro scenari hanno prodotto gli esiti previsti. Sono stati poi raccolti in uno script riutilizzabile, eseguito integralmente in una sessione R riavviata su Linux con quattro Passed. Il codice è nelle pagine successive.

# Script riutilizzabile: preparazione

File del progetto: R/validation/test_validate_fact_sales.R. I blocchi delle pagine 16-18 raccolgono il codice funzionale costruito insieme.

## Caricamento delle funzioni e della demo

```
source(here::here("R", "config", "data_paths.R"))
source(
  here::here("R", "validation", "validate_fact_sales.R")
)
validator_test_paths <- pharmabi_data_paths("demo")
validator_test_source <- readRDS(
  validator_test_paths$fact_file
)
```

Lo script legge il dataset sintetico e carica la funzione reale. Non usa gli oggetti preparati nelle prove manuali. Il dataset demo deve essere già stato generato. La funzione della pagina successiva va definita prima delle chiamate che la usano.

## Preparazione dei tre input errati

```
validator_test_results <- NULL

validator_case_duplicate <- dplyr::bind_rows(
  validator_test_source,
  validator_test_source[1, ]
)

validator_case_profit <- validator_test_source
stopifnot(validator_case_profit$revenue[1] != 0)
validator_case_profit$profit[1] <-
  validator_case_profit$profit[1] + 1
validator_case_profit$margin[1] <-
  validator_case_profit$profit[1] /
  validator_case_profit$revenue[1]

validator_case_margin <- validator_test_source
validator_case_margin$margin[1] <-
  validator_case_margin$margin[1] + 0.10
```

## Perché separare i casi

Ogni input errato deriva dalla stessa copia integra. La duplicazione non viene trascinata nei casi economici e la modifica del profitto non viene trascinata nel caso del margine. Questo permette di associare il rifiuto alla condizione prevista per ciascun scenario.

Nel file già costruito la definizione della funzione può precedere la preparazione dei casi: entrambe devono comparire prima del blocco di esecuzione. I commenti e gli spazi possono differire senza modificare le istruzioni.

# Script riutilizzabile: funzione di prova

La funzione esegue il validatore, intercetta l’eventuale errore e decide se l’esito corrisponde a quello atteso.

```
run_validator_case <- function(
    case_name, test_data, expected_error = NULL
) {
  test_file <- tempfile(fileext = ".rds")
  on.exit(unlink(test_file), add = TRUE)
  saveRDS(test_data, test_file)

  result <- tryCatch(
    {
      validate_fact_sales(test_file)
      list(accepted = TRUE, message = "Validation accepted")
    },
    error = function(e) {
      list(accepted = FALSE, message = conditionMessage(e))
    }
  )

  passed <- if (is.null(expected_error)) {
    result$accepted
  } else {
    !result$accepted &&
      grepl(expected_error, result$message, fixed = TRUE)
  }

  data.frame(
    test = case_name,
    status = if (passed) "Passed" else "Failed",
    expected = if (is.null(expected_error)) {
      "Accepted"
    } else {
      expected_error
    },
    observed = result$message,
    stringsAsFactors = FALSE
  )
}
```

## Come interpretare il risultato

Senza expected_error il test richiede che la validazione termini senza errore. Con expected_error richiede sia il rifiuto sia la presenza del testo previsto nel messaggio. Un errore diverso produce Failed, anziché essere considerato automaticamente un successo.

tryCatch() conserva l’esito o il messaggio nella lista result. on.exit() elimina il file temporaneo quando la funzione termina. La funzione restituisce una riga con nome del test, stato, atteso e osservato.

Il confronto usa una parte del messaggio delle asserzioni attuali. Una modifica ai messaggi o ai nomi interni del validatore può richiedere una revisione dei riferimenti attesi: il test va mantenuto insieme al codice.

# Esecuzione completa e riscontro

I quattro casi vengono eseguiti dalla stessa funzione. I messaggi del validatore restano visibili, mentre il riepilogo mostra gli esiti dei test.

```
validator_test_results <- do.call(
  rbind,
  list(
    run_validator_case("Valid demo", validator_test_source),
    run_validator_case(
      "Duplicate key", validator_case_duplicate,
      expected_error = "duplicate_keys"
    ),
    run_validator_case(
      "Incorrect profit", validator_case_profit,
      expected_error = "max_profit_difference"
    ),
    run_validator_case(
      "Incorrect margin", validator_case_margin,
      expected_error = "max_margin_difference"
    )
  )
)
print(
  validator_test_results[c("test", "status")],
  row.names = FALSE
)
stopifnot(all(validator_test_results$status == "Passed"))
message("All four fact-validator tests passed.")
```

## Verifica dopo il riavvio di R

Dopo aver salvato il file è stata riavviata la sessione R. Nella nuova console è stato eseguito soltanto il comando seguente:

```
source(
  here::here("R", "validation", "test_validate_fact_sales.R")
)
```

## Riepilogo osservato, anche nella nuova sessione

```
test status
       Valid demo Passed
    Duplicate key Passed
 Incorrect profit Passed
 Incorrect margin Passed
All four fact-validator tests passed.
```

Questa esecuzione conferma che lo script carica ciò che gli serve senza dipendere dagli oggetti lasciati in memoria dalle prove precedenti. Richiede comunque il progetto, i pacchetti utilizzati e il dataset demo già disponibile su disco. La nuova batteria di quattro scenari è stata verificata su Linux; l’esecuzione su Windows resta un passaggio distinto dai 70 controlli già eseguiti sui due sistemi.